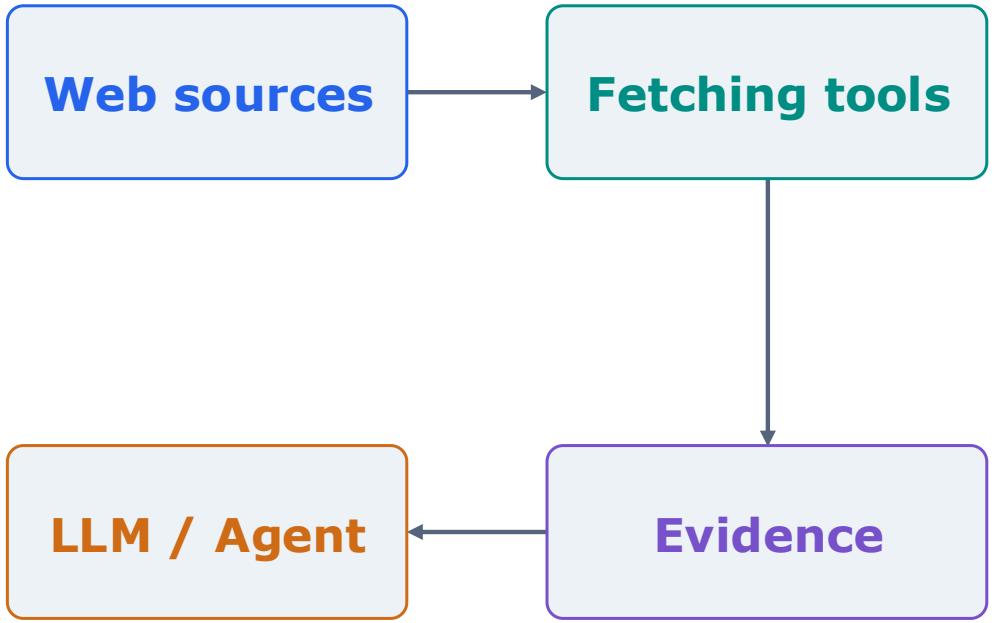
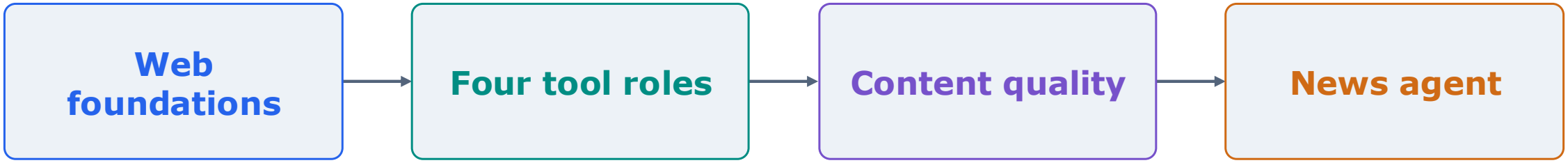
Web Content Fetching for LLMs and AI Agents

BeautifulSoup, Playwright, Crawl4AI and Tavily in practice



Find sources. Retrieve content. Build briefings with evidence.

Workshop roadmap



HTTP, DOM, selectors

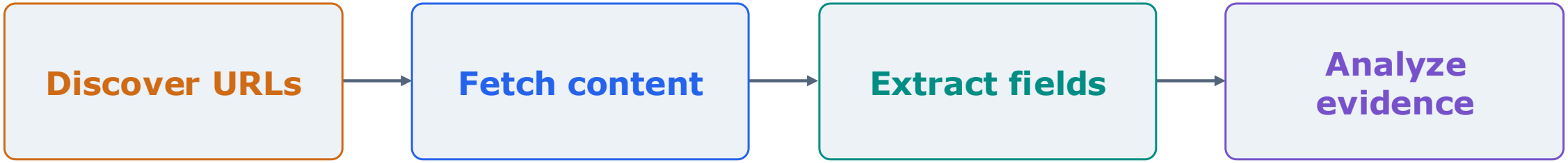
Static and dynamic pages

Markdown, JSON, sources

Briefing and investigation

By the end, choose a fetching route and explain an evidence-based news agent.

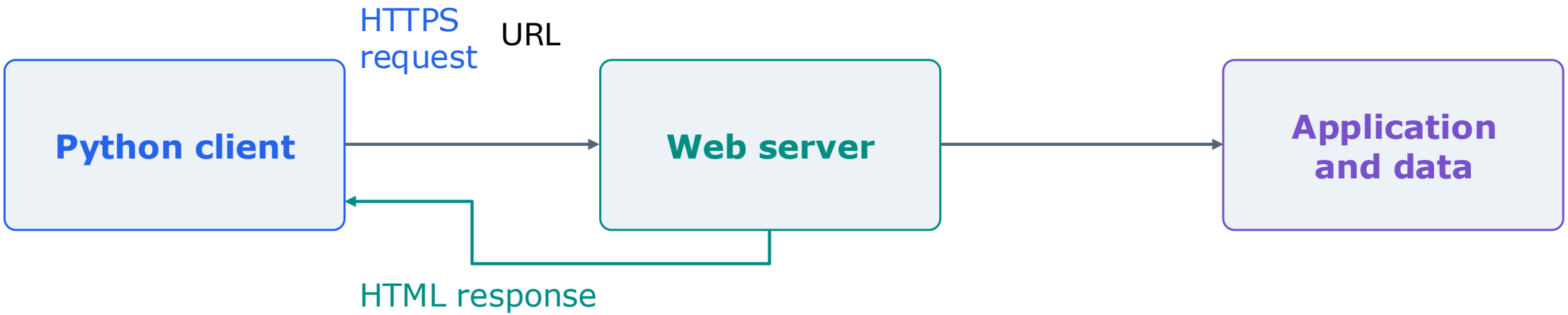
Discovery, fetching and extraction



Discover URLs	Fetch content	Extract fields	Analyze evidence
Topic or source list	HTML or rendered DOM	Article text and metadata	Summary and implications

A search result is a lead. Check the retrieved content before using it as evidence.

How a Python client receives a page



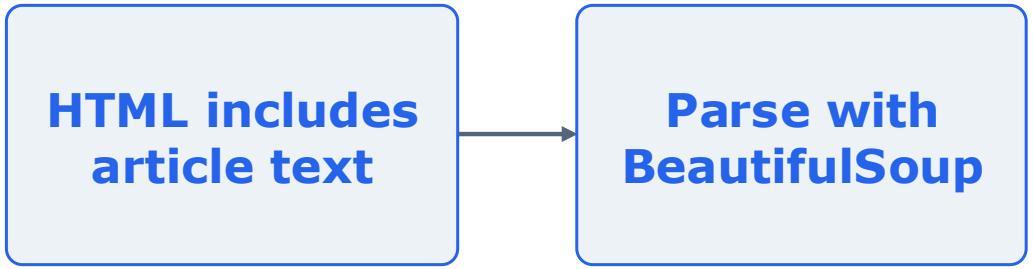
Two separate jobs

Requests retrieves the page. Tools such as BeautifulSoup could extract fields from its HTML.

The HTTP response contains the initial document, which may differ from the rendered page.

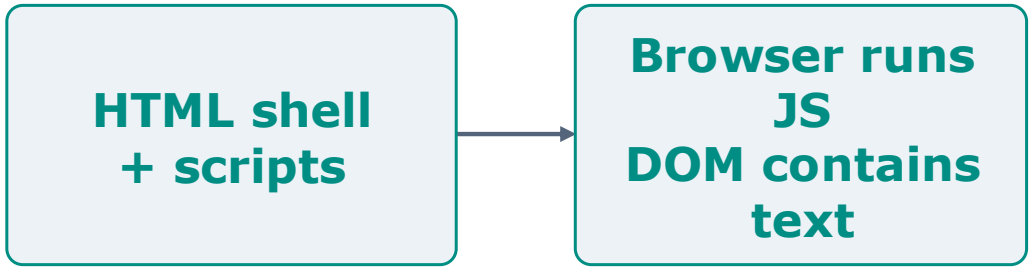
Initial HTML and the rendered DOM

Server-rendered content



Inspect response.text

Content added by JavaScript



Inspect browser DOM

Inspect the response and the live DOM before selecting a tool.

HTML anatomy for extraction

```
<html>
  <head><title>Industry News</title></head>
  <body>
    <article class="story" id="a17">
      <h1>New AI service launched</h1>
      <time datetime="2026-10-31">Today</time>
      <p>The company announced...</p>
      <a href="/story/17">Read more</a>
    </article>
  </body>
</html>
```

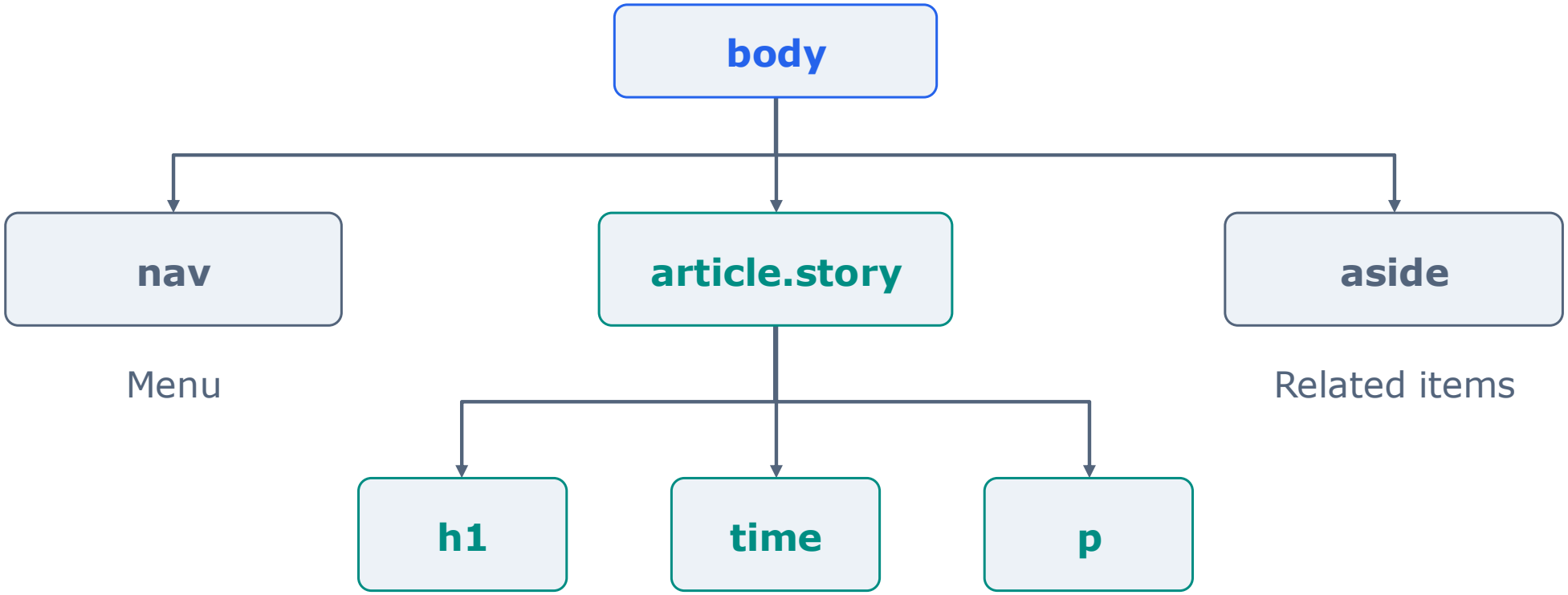
h1
Headline

time[datetime]
Publication time

a[href]
Next article URL

Look for meaning and structure: title, article body, timestamp and links.

The DOM is a tree



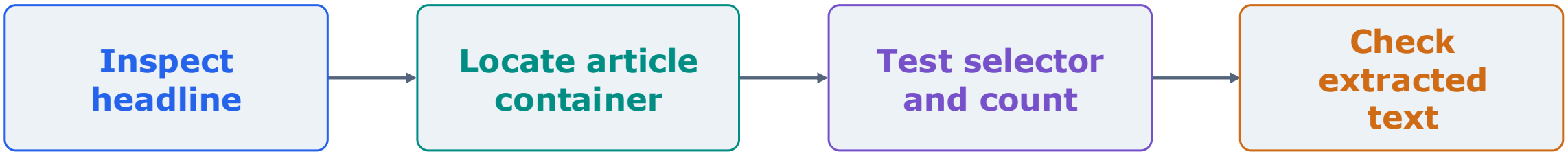
Scope extraction to the article so navigation and related stories stay outside it.

CSS selectors connect code to content

Selector	Meaning	Example target
`article`	Tag	Main story container
`.story`	Class	Elements with class="story"
`#a17`	ID	Element with id="a17"
`article.story h1`	Descendant	Headline inside an article
`a[href]`	Attribute	Links with a URL

Use stable semantic structure and verify that the selector returns the intended items.

Inspect, select and verify




```
article.story h1    → 1 headline
article.story p     → article paragraphs
nav a               → menu links (exclude)
```

A selector is only useful when the extracted text is the content you intended.

Static fetching with Requests

```
import requests
from bs4 import BeautifulSoup

r = requests.get(URL, timeout=20)
r.raise_for_status()
soup = BeautifulSoup(r.text, "lxml")

print(r.status_code)
print(r.headers.get("Content-Type"))
```

01 Response status and type

02 Encoding and readable text

03 Expected article present

Check status, timeouts and encoding before parsing the response.

BeautifulSoup selects article fields

```
article = soup.select_one("article.story")
if article is None:
    raise ValueError("Article missing")

headline = article.select_one("h1")
paragraphs = article.select("p")

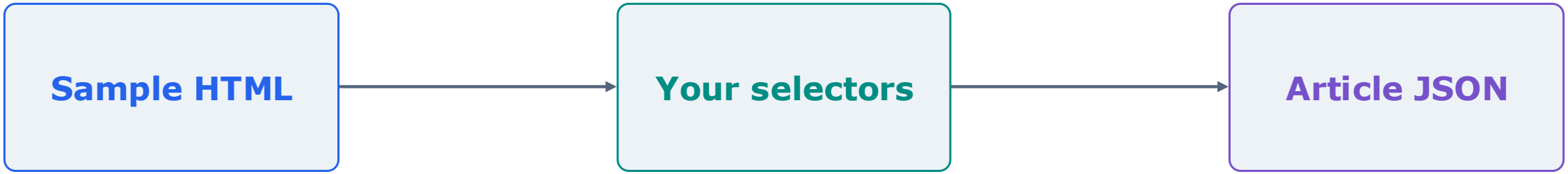
title = (headline.get_text(" ", strip=True)
        if headline else None)
body = "\n".join(
    x.get_text(" ", strip=True)
    for x in paragraphs
)
```

Structured record

title
body
source_url
published_at

Missing fields should remain explicit rather than being invented.

Exercise 1: extract a news article

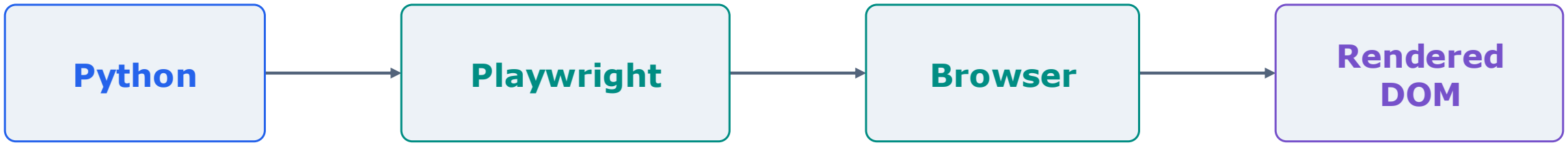


Extract and check

Title, publication time, article body and URL. Confirm menu text is excluded.

Expected result: one validated article record with a traceable source.

Playwright controls a real browser



Browser actions

Navigate, wait for content, click, paginate and read text.

Use browser automation when content rendering or interaction requires it.

Wait for the content you need

```
await page.goto(URL,
    wait_until="domcontentloaded")

article = page.locator("article.story")
await article.wait_for(state="visible")

headline = await article.locator("h1").inner_text()
body = await article.inner_text()
html = await page.content()
```

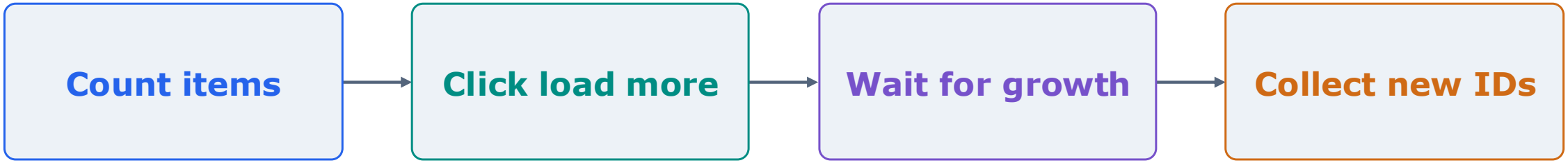
01 Initial HTML parsed

02 Target element visible

03 Expected content verified

Action readiness and article completeness are separate checks.

Interaction extends what can be fetched



Stop when: no next page, no new IDs, or the configured page limit is reached.

Demo: browser content to saved HTML

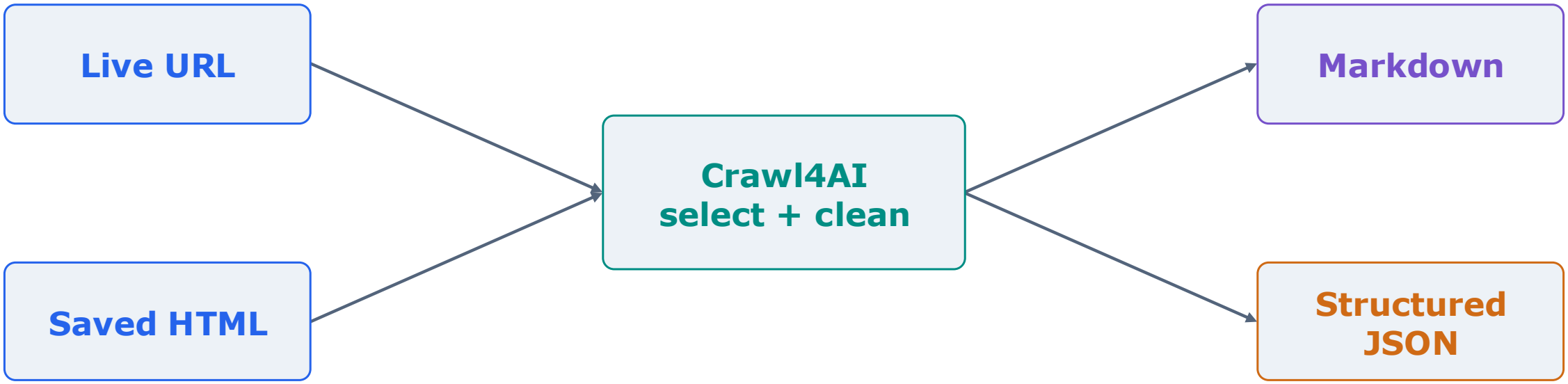


raw.html

Reusable input for BeautifulSoup or Crawl4AI

Use async in Jupyter. Use sync in a standalone script without a running event loop.

Crawl4AI prepares content for analysis



Crawl4AI can fetch pages itself. A separate Playwright step is optional.

Targeted Markdown reduces page noise

```
md = DefaultMarkdownGenerator()
config = CrawlerRunConfig(
    target_elements=["article.story"],
    word_count_threshold=1,
    markdown_generator=md
)

result = await crawler.arun(
    url=URL, config=config
)
```

Before

Menus + adverts +
article
+ related stories

After

Selected article
with headings and links

Readable formatting helps the model, but does not guarantee complete or accurate extraction.

Structured JSON without an LLM

```
schema = {
  "name": "articles",
  "baseSelector": "article.story",
  "fields": [
    {"name": "title", "selector": "h1",
     "type": "text"},
    {"name": "body", "selector": ".body",
     "type": "text"}
  ]
}
strategy = JsonCssExtractionStrategy(schema)
```

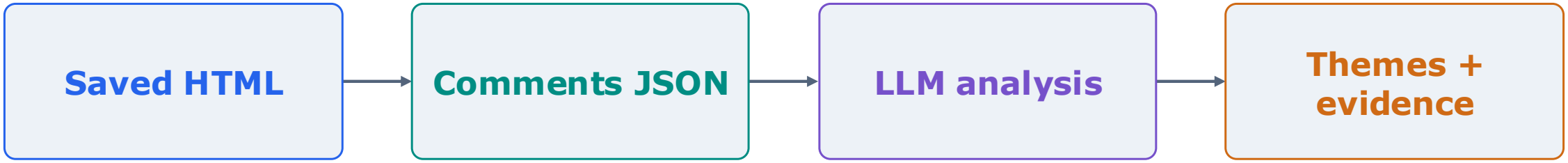
HTML structure

Schema

JSON records

Deterministic extraction is useful for stable fields. Validate the returned values.

Demo: extracted comments to an LLM summary



Transfer the pattern to news

Replace comment fields with headline, article body, publication time and source URL.

Separate collection and extraction from the model’s interpretation.

Tavily discovers sources from a topic



Search intent

Example: recent enterprise AI adoption announcements in selected markets.

Discovery extends coverage. It does not prove that every relevant article was found.

Tavily capabilities and retrieval routes

Capability	Input	Output / role
Search	Topic or question	Ranked candidate sources
Extract	Known URLs	Retrieved page content
Map	Starting website	Discovered site URLs
Crawl	Starting website	Traversed pages and content

Two valid patterns: Search + Extract, or Search + your own fetching tools.

Choose the service and extraction route that gives the required evidence and control.

Demo: discover, select and retrieve

```
hits = tv.search(
    query="enterprise AI adoption news",
    search_depth="advanced",
    max_results=10
)

for hit in hits["results"]:
    print(hit["title"], hit["url"])
```

01 Select relevant sources

02 Retrieve full usable content

03 Retain URLs and dates

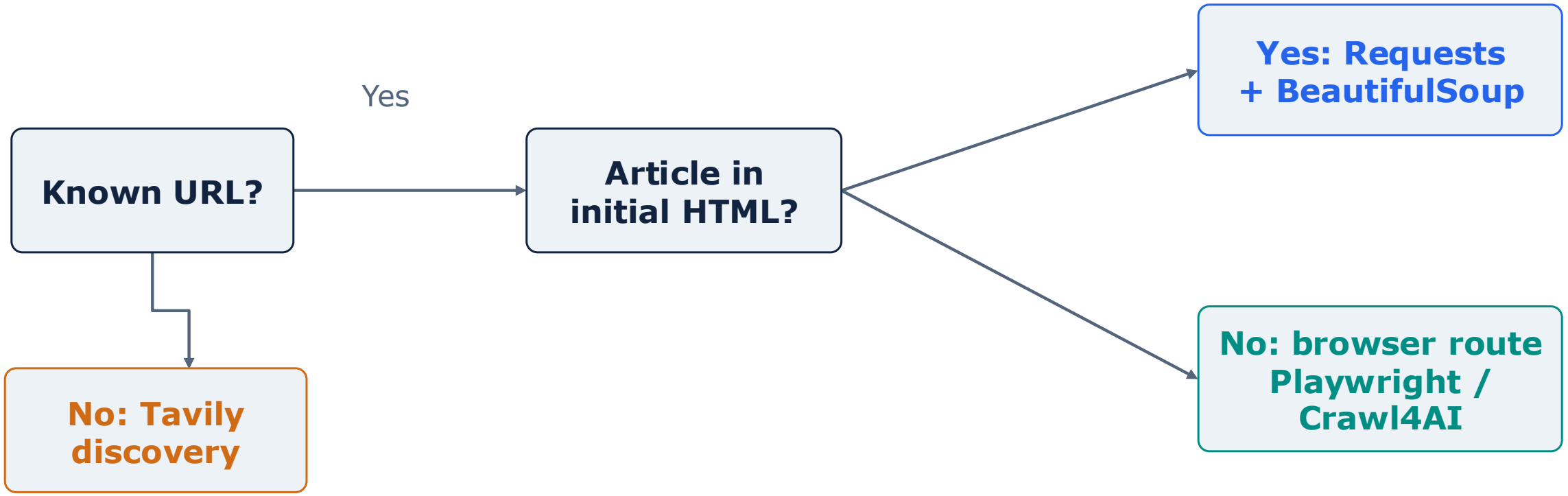
Inspect the article text before adding it to a summary prompt.

Tool selection by task

Need	Typical route	Why
Static article HTML	Requests + BeautifulSoup	Direct and easy to inspect
Browser interaction	Playwright	Rendering and actions
Markdown or schema output	Crawl4AI	Content preparation
Topic-based discovery	Tavily Search	Find candidate URLs
Managed URL retrieval	Tavily Extract	Service-based extraction

The tools overlap. Select a route rather than forcing every page through all four.

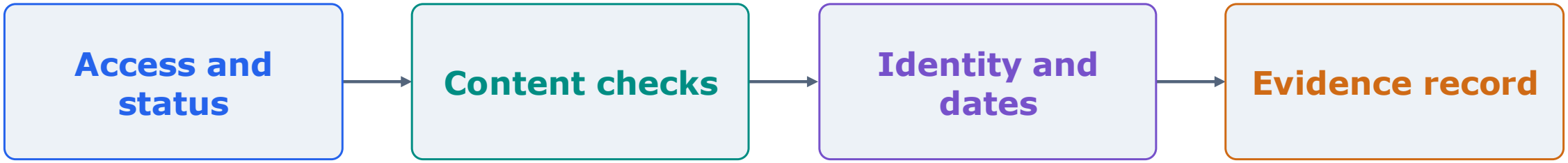
A practical routing decision



Need managed extraction? Tavily Extract is another option.

Prefer the simplest route that retrieves the required content reliably.

A quality gate before the LLM

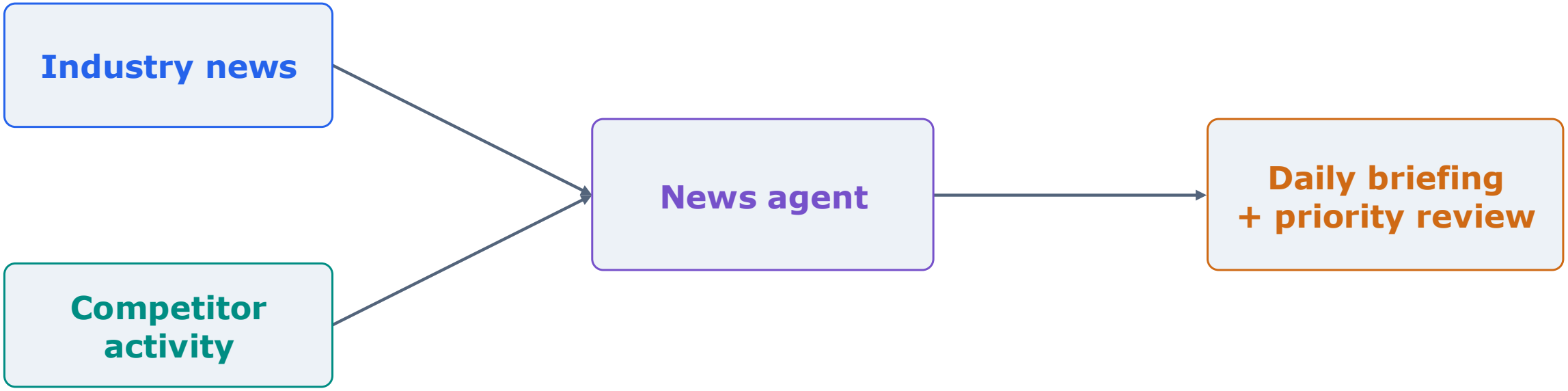


Access and status	Content checks	Identity and dates	Evidence record
Timeouts, retries, rate limits	Body present, not a login page	Duplicates, language, timestamp	URL, retrieval time, method

Cache usable content. Log failures. Keep page instructions outside agent authority.

HTTP 200 alone does not prove that a usable article was collected.

Case: a daily industry intelligence briefing

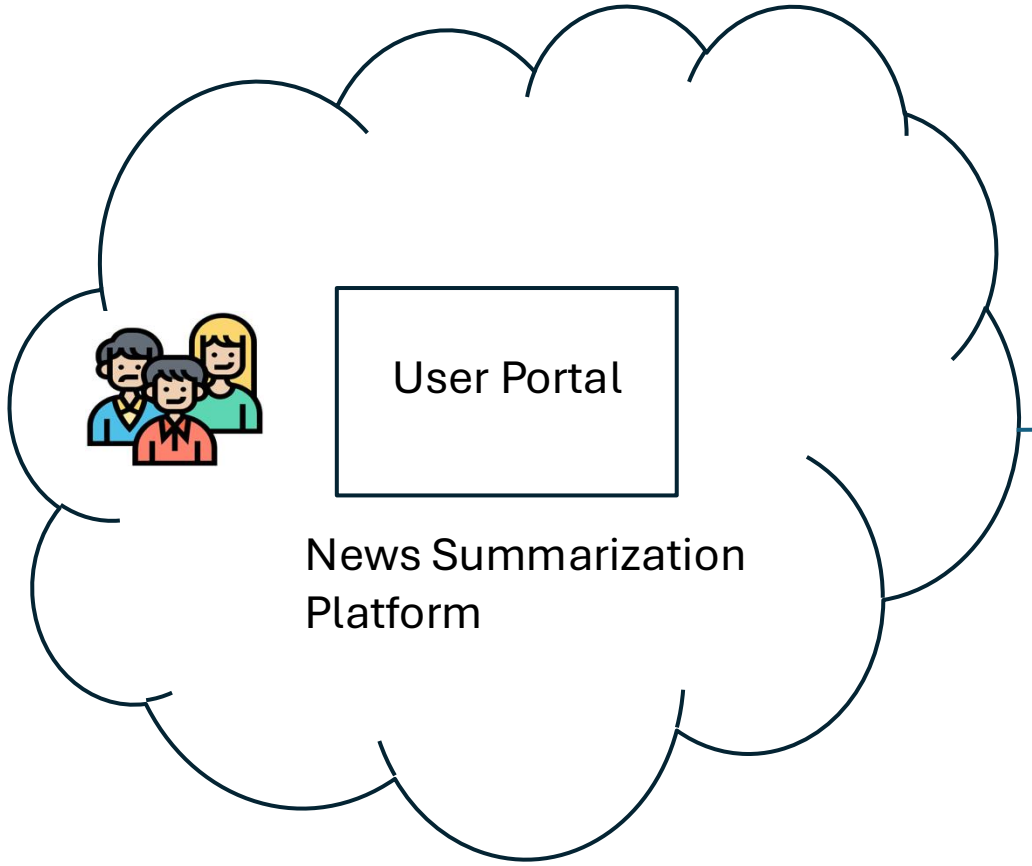


Approved sources • Defined topics • Reporting window

The goal is a relevant, traceable briefing with clear gaps and material changes.

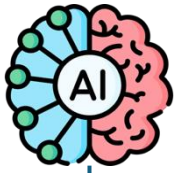
Solution Framework

-Centralized news fetching & storage
-Dashboard showing insights and trends of different news topic
-Personalized news subscription
-Chatbot for user query and analysis



API

LLM



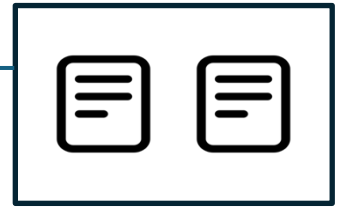
Agentic Framework

Tools

Playwright

Crawl4ai

tavily by NEBIUS



Document DB

Admin Portal

<u>Centralized Backbone</u>

News Repository Agent

News Content Fetching Workflow

Prompts/Keywords

Tools calling

Playwright

Crawl4ai

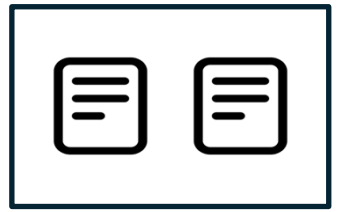
tavily by NEBIUS

Topic

News Content Fetching

News Content validation & Parsing

Document Upload (DB)

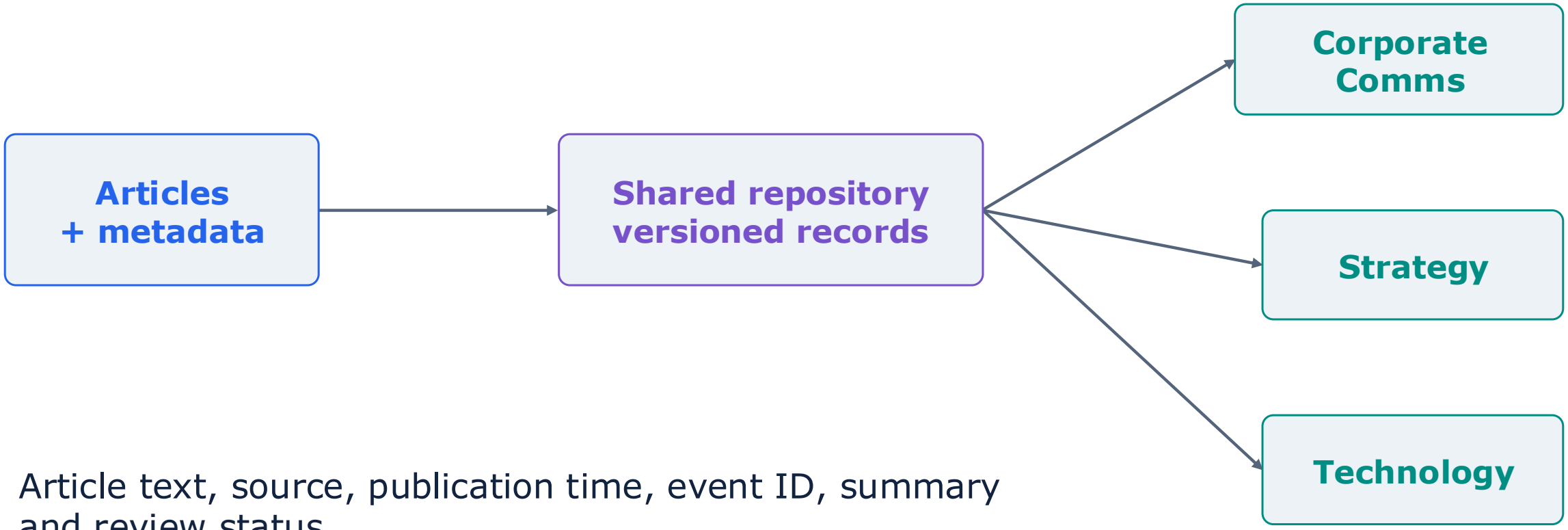


Daily News content From different news media

類別	傳媒名稱	新聞頁網址
電子傳媒	無綫新聞	https://news.tvb.com
電子傳媒	Now新聞	https://news.now.com
電子傳媒	有綫新聞	https://www.i-cable.com
電子傳媒	商業電台	https://www.881903.com
電子傳媒	香港電台	https://news.rthk.hk
電子傳媒	新城電台	https://www.metroradio.com.hk/news/
印刷媒體	am730	https://www.am730.com.hk
印刷媒體	明報	https://www.mingpao.com/
印刷媒體	星島日報	https://www.stheadline.com
印刷媒體	頭條日報	https://www.stheadline.com
印刷媒體	東方日報	https://www.on.cc
印刷媒體	香港經濟日報	https://www.hket.com
印刷媒體	信報	https://www.hkej.com
印刷媒體	文匯報	https://www.wenweipo.com
印刷媒體	大公報	https://www.takungpao.com
印刷媒體	南華早報	https://www.scmp.com
印刷媒體	英文虎報 (The Standard)	https://www.thestandard.com.hk
印刷媒體	成報	https://www.singpao.com.hk
印刷媒體	香港商報	https://www.hkcd.com.hk
印刷媒體	紫荊雜誌社	https://www.bau.com.hk
網絡媒體	AASTOCKS	https://www.aastocks.com
網絡媒體	Bastille Post	https://www.bastillepost.com
網絡媒體	Capital	https://www.capital-hk.com
網絡媒體	ET Net	https://www.etnet.com.hk
網絡媒體	HK01	https://www.hk01.com
網絡媒體	i-Money	https://imoney.hket.com
網絡媒體	Infocast	https://www.infocastfn.com
網絡媒體	Job Market	https://www.jobmarket.com.hk
網絡媒體	Quamnet	https://www.quamnet.com
網絡媒體	Recruit	https://www.recruit.com.hk
網絡媒體	Yahoo! 新聞	https://hk.news.yahoo.com
網絡媒體	晴報	https://www.skypost.hk
網絡媒體	圈傳媒	https://www.looop.hk
網絡媒體	堅料網	https://www.kinliu.hk/
網絡媒體	經濟一週	https://www.edigest.hk/
網絡媒體	Sunday Kiss	https://www.sundaykiss.com/
網絡媒體	點新聞	https://www.dotdotnews.com
網絡媒體	橙新聞	https://www.orangenews.hk
網絡媒體	投見名家	https://www.masters-insight.com

Reference news media sources

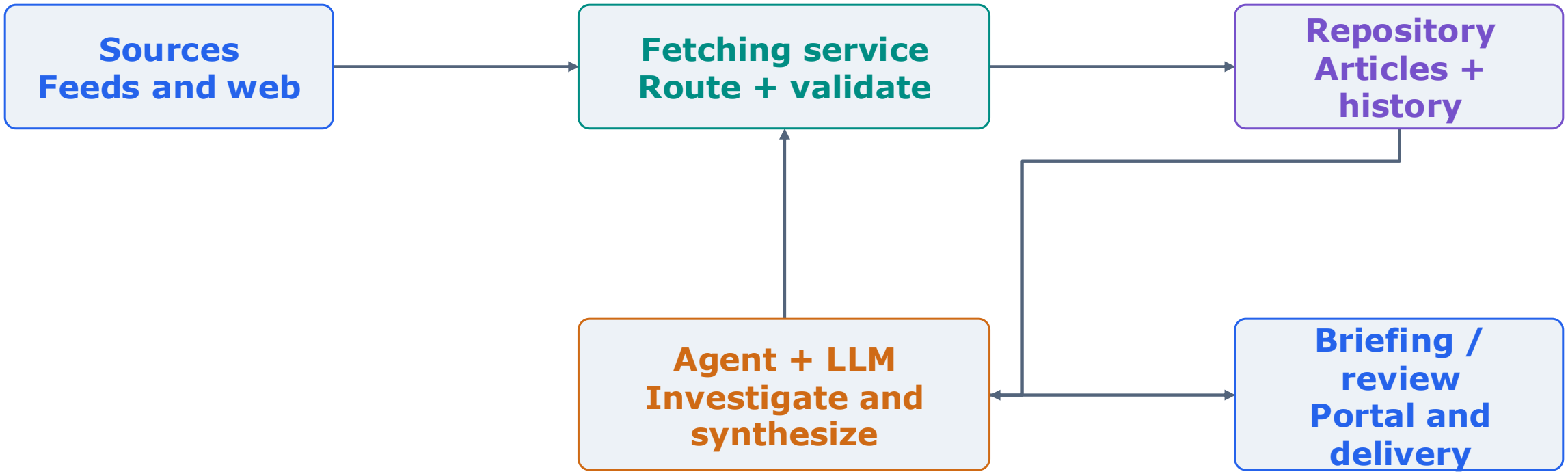
One repository supports many audiences



Article text, source, publication time, event ID, summary and review status

Reuse collected evidence and approved summaries while preserving departmental context.

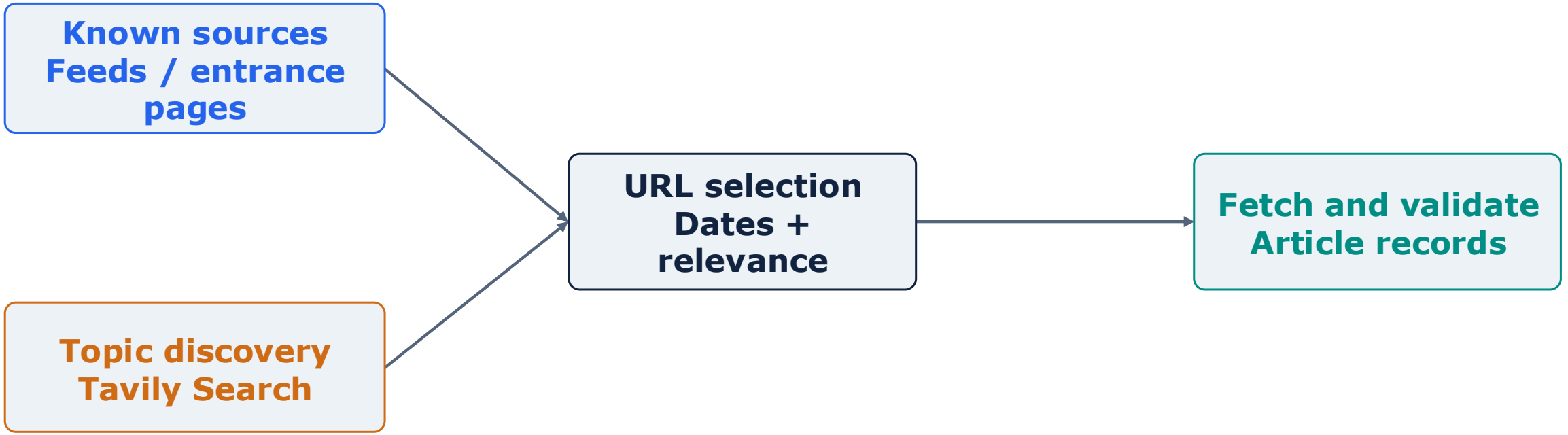
News-agent solution architecture



Tools: Requests / BeautifulSoup, Playwright, Crawl4AI, Tavily

Keep the collection service predictable and use the agent for evidence-dependent decisions.

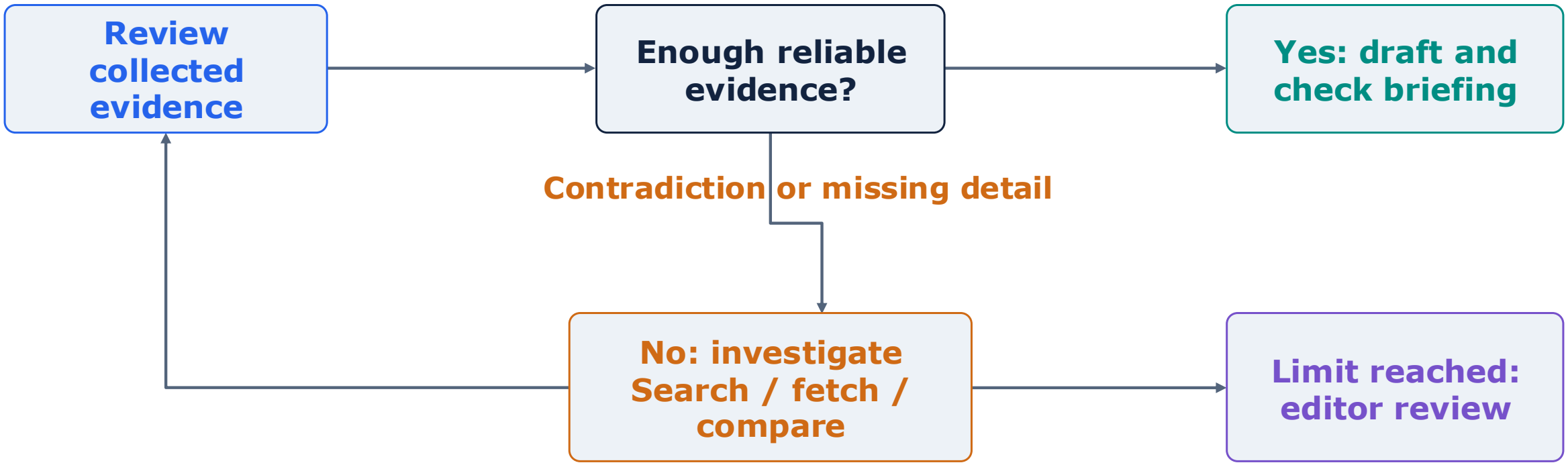
Collection has two entry routes



Then: duplicate detection, event grouping, storage and incremental updates

Combine predictable source monitoring with targeted discovery for evidence gaps.

The agent investigates before completing



Stop when evidence checks pass, or hand over unresolved issues within defined limits.

From article summaries to useful insights

Illustrative briefing entry

Competitor launches an AI service

FACT
Two reports quote $20m; the official announcement states $12m.

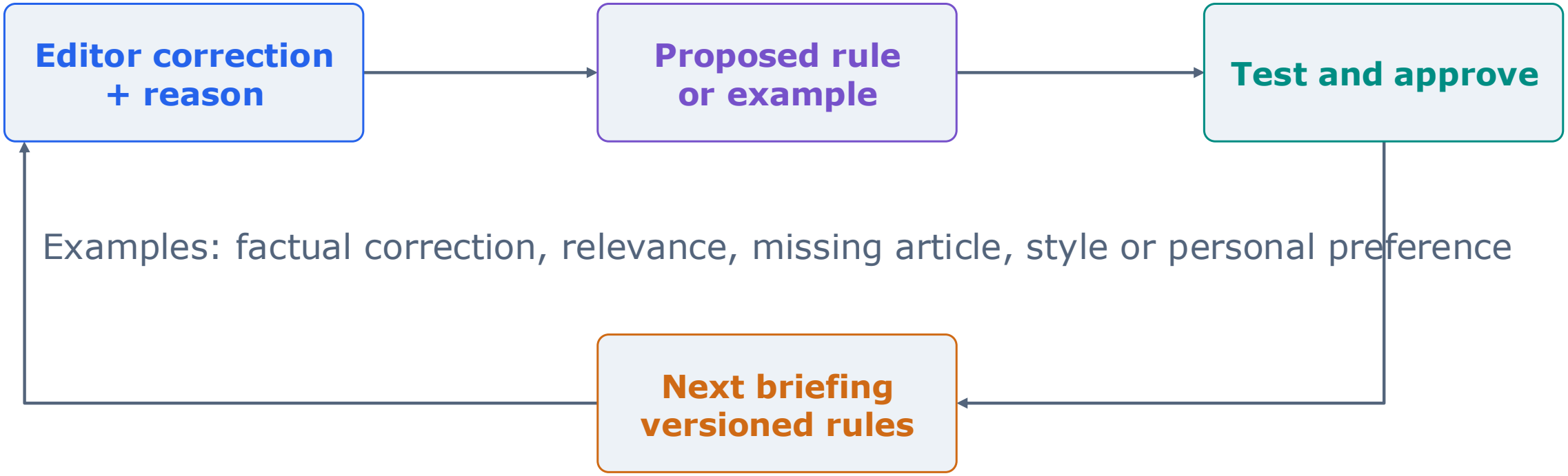
INTERPRETATION
The reporting contains a material discrepancy.

Recommended follow-up

Use the official figure.
Flag the discrepancy for review.
Keep source links attached.

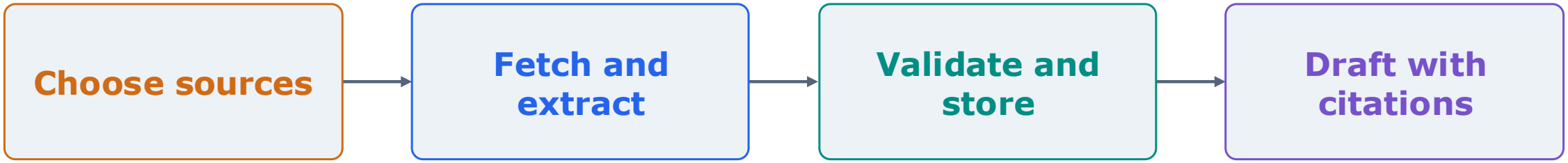
Separate reported facts, the agent’s interpretation and the recommended follow-up.

Editorial feedback improves the next briefing



Rule and example updates are tested and approved before being applied.

Final exercise: assemble a news briefing



Deliverables

Three article records, a short briefing, one evidence gap, and a diagram of your route.

Deliver a small evidence-backed briefing and explain every tool choice.

A practical fetching toolkit

Tool	Core contribution	Use in the news agent
BeautifulSoup	Parse known HTML structures	Article fields and links
Playwright	Render and interact	Dynamic article access
Crawl4AI	Prepare Markdown and JSON	Clean, structured content
Tavily	Discover and retrieve sources	Topic search and follow-up
LLM / Agent	Interpret and decide next steps	Briefing and investigation

Reliable news intelligence starts with reliable evidence and explicit completion checks.